

10

/ 13 LECTURES

TCP, UDP and Socket Programming

The transport layer: ports and multiplexing, TCP vs UDP, the three-way handshake and four-way close, reliability, flow control, and real socket code in Java and C++.

Ports & Multiplexing

UDP — 8-byte header

TCP — 4 guarantees

3-Way Handshake

4-Way Close · TIME_WAIT

ACK & Retransmission

Sliding Window

Sockets — Java & C++

Client-Server Model

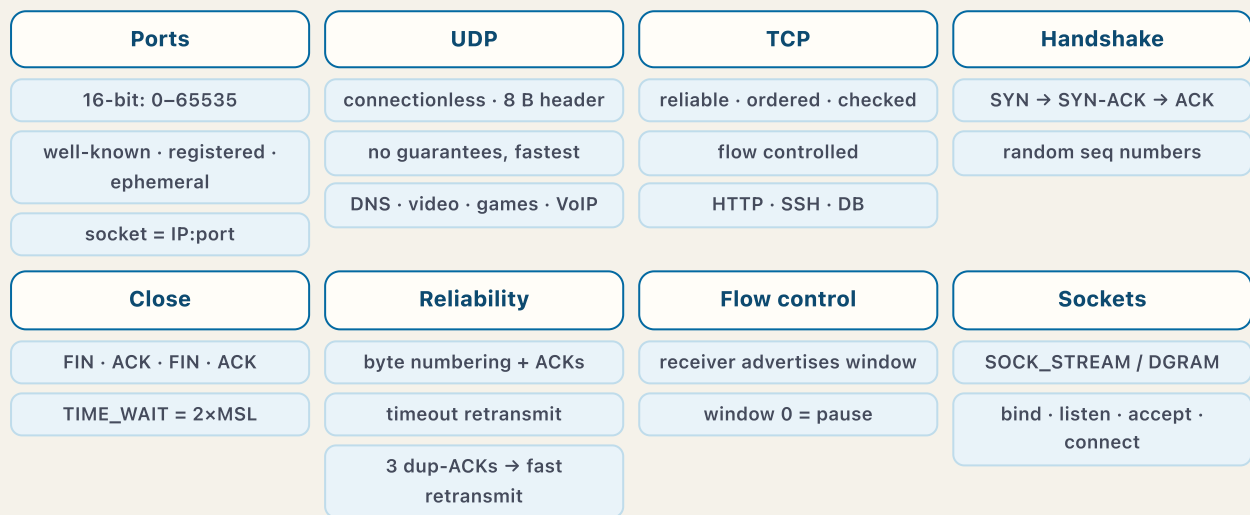
WHAT THIS LECTURE GIVES YOU

How one machine runs a hundred apps at once (ports), the full TCP-vs-UDP decision framework with real applications, byte-level handshake and close sequences, how TCP recovers losses (ACKs, timeouts, fast retransmit), how the sliding window prevents flooding, and working TCP/UDP client-server code in Java and C++ you can run tonight.

LEARNING OBJECTIVES — AFTER THIS LECTURE YOU CAN

- Explain the transport layer's role: **ports**, **multiplexing**, **demultiplexing**
- Compare **TCP and UDP** and pick correctly per application
- Trace the **three-way handshake** and **four-way close** (with TIME_WAIT)
- Explain TCP's **reliability**: ACKs, RTO, fast retransmit; and **flow control**: sliding window
- Write **TCP/UDP client-server programs** in Java or C++

Transport Layer



1 Ports — Many Apps, One Machine

IP delivers to a **machine**; a machine runs many programs. A **port** (16-bit, 0–65535) identifies the process. Full address = **IP : port** (e.g. 192.168.1.5:52341). **Multiplexing** tags outgoing packets with source ports; **demultiplexing** delivers incoming packets to the right process by destination port.

Range	Name	Use	Range	Name	Use
0–1023	Well-known	HTTP 80 · HTTPS 443 · SSH 22 · DNS 53 · SMTP 25	1024–49151	Registered	MySQL 3306 · Redis 6379 · Postgres 5432
49152–65535	Ephemeral	Client-side temporary ports	— the pair (src port, dst port) identifies the conversation		

2 UDP — Fast and Fearless

Src Port 16 bit

Dst Port 16 bit

Length 16 bit

Checksum 16 bit

Figure 1. The entire UDP header — 8 bytes. Then data. That's it.

Application	Why UDP
DNS	single request-response; latency matters more than ceremony
Video streaming	an old frame is worthless — skip > wait
Online gaming	position updates are time-critical; retransmitting stale ones hurts
VoIP calls	one lost packet = a tiny glitch, not a paused call
DHCP	needs broadcast; TCP can't broadcast
QUIC / HTTP-3	UDP base + custom reliability in user space

3 TCP — Reliable, Ordered, Controlled

TCP's four guarantees

Reliable (data arrives or sender knows) · **In-order** (bytes arrive as sent) · **Error-checked** (bad segments retransmitted) · **Flow-controlled** (never drowns the receiver).

Key header fields

Src/Dst Port (16b) · **Sequence** (32b — byte position!) · **Acknowledgment** (32b — next byte expected) · **Flags** (SYN ACK FIN RST PSH URG) · **Window** (16b — receiver buffer) · Checksum.

Three-way handshake (connection setup)



Figure 2. SYN → SYN-ACK → ACK. Why random initial sequence numbers? So stale packets from an old connection can't masquerade as new ones.

Four-way close (connection teardown)

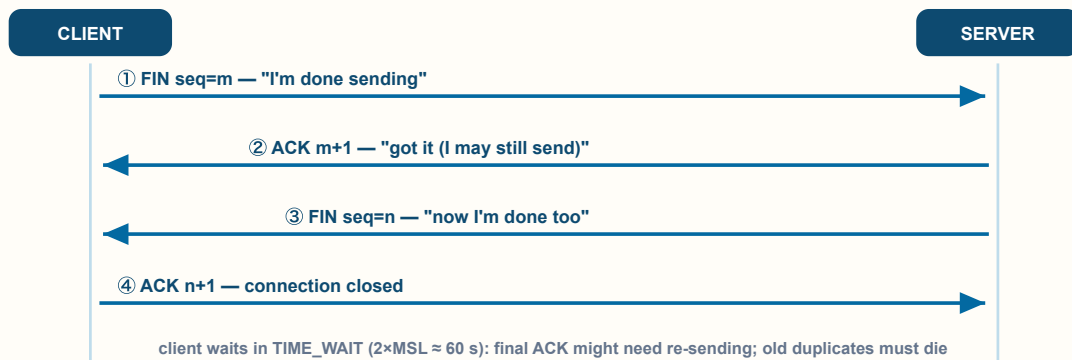


Figure 3. Each side closes independently — FIN/ACK in each direction.

Reliability — ACKs and retransmission

```
Sender: [seq 1–1000] [seq 1001–2000] [seq 2001–3000]
Receiver: [ACK 1001] [ACK 2001] x LOST
Then: [ACK 3001 arrives anyway...]
Receiver keeps repeating: [ACK 2001] [ACK 2001] [ACK 2001] ← 3 duplicate ACKs
Sender: FAST RETRANSMIT seq 2001 immediately (no waiting for timeout)
```

Two retransmission triggers

1. RTO timeout — no ACK within the retransmission timeout → resend. **2. Fast retransmit** — 3 duplicate ACKs → resend immediately (much faster).

Flow control — sliding window

Every ACK carries a **window size** = "send up to N unacknowledged bytes". Big buffer → window 65535 → pipeline freely. Full buffer → **window 0** → **sender pauses** (zero window).

4 Socket Programming — Java & C++

A **socket** is the OS's endpoint abstraction: `AF_INET` (IPv4) × `SOCK_STREAM` (TCP) or `SOCK_DGRAM` (UDP).

```
SERVER                                CLIENT
socket()                             socket()
bind(('0.0.0.0', 8080))
listen(backlog)
accept() — 3-way — connect()
recv() ← data — send()
send() — reply —> recv()
close() — 4-way —> close()
```

Figure 4. The client-server lifecycle — every socket program follows this dance.

TCP ECHO SERVER

JAVA

```
import java.io.*; import java.net.*;

try (ServerSocket server = new
ServerSocket(8080)) { // bind + listen
    System.out.println("listening on 8080");
    while (true) {
        Socket c = server.accept();
        // blocks – handshake happens here
        var in = new BufferedReader(
            new
InputStreamReader(c.getInputStream()));
        var out = new
PrintWriter(c.getOutputStream(), true);
        String line = in.readLine();
        out.println("Echo: " + line);
        c.close();
    }
}

// 4-way close
}
```

TCP ECHO SERVER

C++

```
int server = socket(AF_INET, SOCK_STREAM, 0);
int yes = 1;
setsockopt(server, SOL_SOCKET, SO_REUSEADDR,
&yes, sizeof(yes));
sockaddr_in addr{AF_INET, htons(8080),
{INADDR_ANY}};
bind(server, (sockaddr*)&addr, sizeof(addr));
listen(server, 5);
// backlog of 5
printf("listening on 8080\n");

while (true) {
    int c = accept(server, nullptr, nullptr);
    // blocks – handshake happens here
    char buf[1024] = {0};
    read(c, buf, sizeof(buf));
    string reply = string("Echo: ") + buf;
    write(c, reply.c_str(), reply.size());
    close(c);
}

// 4-way close
}
```

TCP CLIENT + UDP PAIR

JAVA

```
// TCP client – the handshake happens inside
connect()
try (Socket client = new Socket("127.0.0.1",
8080)) {
    var out = new
PrintWriter(client.getOutputStream(), true);
    var in = new BufferedReader(new
InputStreamReader(client.getInputStream()));
    out.println("Hello!");
    System.out.println(in.readLine());
}

// 4-way close

// UDP server – no listen/accept at all
try (DatagramSocket u = new
DatagramSocket(9090)) {
    byte[] buf = new byte[1024];
    DatagramPacket p = new
DatagramPacket(buf, buf.length);
    u.receive(p);
    // + sender address
    u.send(new DatagramPacket("got
it".getBytes(), 6,
                                p.getAddress(),
                                p.getPort()));
}

// UDP client
try (DatagramSocket uc = new
DatagramSocket()) {
    uc.send(new
DatagramPacket("ping".getBytes(), 4,

InetAddress.getByName("127.0.0.1"), 9090));
    byte[] buf = new byte[1024];
    DatagramPacket r = new
DatagramPacket(buf, buf.length);
    uc.receive(r);
    System.out.println(new
String(r.getData(), 0, r.getLength()));
}
```

TCP CLIENT + UDP PAIR

C++

```
// TCP client – the handshake happens inside
connect()
int c = socket(AF_INET, SOCK_STREAM, 0);
sockaddr_in a{AF_INET, htons(8080),
{inet_addr("127.0.0.1")}};
connect(c, (sockaddr*)&a, sizeof(a));
// <- handshake
write(c, "Hello!", 6);
char buf[1024] = {0};
read(c, buf, sizeof(buf));
close(c);
// 4-way close

// UDP server – no listen/accept at all
int u = socket(AF_INET, SOCK_DGRAM, 0);
sockaddr_in ua{AF_INET, htons(9090),
{INADDR_ANY}};
bind(u, (sockaddr*)&ua, sizeof(ua));
char buf[1024]; sockaddr_in from; socklen_t
fl = sizeof(from);
recvfrom(u, buf, sizeof(buf), 0,
(sockaddr*)&from, &fl); // + sender addr
sendto(u, "got it", 6, 0, (sockaddr*)&from,
fl);

// UDP client
int uc = socket(AF_INET, SOCK_DGRAM, 0);
sockaddr_in dst{AF_INET, htons(9090),
{inet_addr("127.0.0.1")}};
sendto(uc, "ping", 4, 0, (sockaddr*)&dst,
sizeof(dst));
recvfrom(uc, buf, sizeof(buf), 0, nullptr,
nullptr);
```

NOTE `recv()` returning **0 bytes** on a TCP socket = the other side closed cleanly (this is how you detect FIN). To serve many clients at once, wrap each accepted client in a **thread** (or use async selectors).

5 TCP vs UDP — the Decision Table

Feature	TCP	UDP
Connection	Connection-oriented (handshake)	Connectionless — just send
Reliability	Guaranteed (ACK + retransmit)	None — may lose, dup, reorder
Ordering	Guaranteed	Not guaranteed
Error detection	Checksum + retransmit	Checksum only (detects, drops)
Header size	20+ bytes	8 bytes
Speed / overhead	Slower	Faster
Flow control	Yes (window)	No
Use cases	HTTP(S), SSH, email, databases	DNS, streaming, games, VoIP, DHCP, QUIC

EXAM "A live game leaderboard updates every 100 ms — TCP or UDP?" **UDP**: a missed update is stale in 100 ms anyway; retransmission adds nothing but delay. "File download?" **TCP** — every byte must arrive, in order.

LECTURE 10 — RAPID REVISION CARD

Port = 16-bit process ID; socket = IP:port; mux/demux by ports.

UDP: 8-B header, connectionless, no guarantees — DNS, video, VoIP, games, DHCP, QUIC.

TCP: reliable, ordered, error-checked, flow-controlled — HTTP, SSH, DBs.

Handshake: SYN → SYN-ACK → ACK (1 RTT); random ISNs defeat old-packet confusion.

Close: FIN, ACK, FIN, ACK; TIME_WAIT = 2×MSL for lost-ACK safety.

Retransmit: RTO timeout OR 3 duplicate ACKs (fast retransmit).

Window: receiver-advertised unacked byte budget; 0 = pause.

Sockets: TCP bind→listen→accept ↔ connect; UDP just bind→recvfrom/sendto.

recv() == 0 → peer closed. Well-known: 22 · 53 · 80 · 443 · 3306.

Important Questions — Exam Practice

A. Multiple Choice

Q1. The UDP header is exactly:

MCQ

- a) 20 bytes
- b) 8 bytes
- c) 4 bytes
- d) 16 bytes

Q2. In the three-way handshake, packet 2 carries flags:

MCQ

- a) SYN only
- b) ACK only
- c) SYN+ACK
- d) FIN+ACK

Q3. Fast retransmit triggers after:

MCQ

- a) 1 lost packet
- b) RTO expiry
- c) 3 duplicate ACKs
- d) window = 0

Q4. A window size of 0 means:

MCQ

- a) connection closed
- b) sender must pause (receiver buffer full)
- c) maximum speed
- d) UDP fallback

Q5. Which socket call exists ONLY on the TCP server side?

MCQ

- a) socket()
- b) bind()
- c) listen()/accept()
- d) sendto()

Q6. TIME_WAIT lasts $2 \times \text{MSL}$ mainly to:

MCQ

- a) speed up closing
- b) let a lost final ACK be answered by FIN-retransmit, and old duplicates die
- c) free the port instantly
- d) encrypt remaining data

B. Short Answer (2–4 marks)

Q7. What problem do ports solve? Define multiplexing and demultiplexing.

SHORT

Q8. Why does TCP randomise initial sequence numbers?

SHORT

Q9. Why does the close need FOUR messages while open needs three?

SHORT

Q10. Difference between timeout retransmission and fast retransmit — which is faster and why?

SHORT

Q11. What does the Window Size field accomplish? What happens at 0?

SHORT

Q12. Your `recv()` returns `b''` — what happened, and what should your code do?

SHORT

C. Long Answer / Code (5–8 marks)

Q13. Draw and explain the three-way handshake and four-way close with sequence/acknowledgment numbers at every step. Include why TIME_WAIT exists.

LONG

Q14. Write a TCP echo server and its client in Java or C++ (any correct form). Annotate where the handshake and close occur.

CODE

Q15. Write a UDP server-client pair where the client sends a message and prints the server's reply. Why is there no accept()?

CODE

Q16. "TCP for correctness, UDP for speed." Justify with six applications (three each) and the mechanism differences behind the choice.

LONG

ANSWER KEY

A1. b — 4 fields × 2 bytes.

A2. c — server's SYN+ACK.

A3. c — three dup-ACKs beat the timer.

A4. b — zero window = pause until reopened.

A5. c — listen/accept are TCP-server-only; UDP has bind→recvfrom.

A6. b — reliability of the close + duplicate expiry.

A7. Many processes per host; mux = tag outgoing with src port; demux = deliver by dst port.

A8. Prevents old-connection segments being accepted as valid data in a new connection (also raises spoofing difficulty).

A9. Each direction closes independently (half-close): FIN+ACK each way = 4 messages; open needs only agreement = 3.

A10. Timeout = wait for a full RTO; fast retransmit fires on 3 dup-ACKs, usually far sooner.

A11. Receiver advertises free buffer → bounds unacked in-flight bytes; 0 pauses sender.

A12. Peer sent FIN (closed cleanly) — close your socket / stop reading.

A14/A15. Compare with the §4 code: server thread-per-client with accept() in a loop; UDP has no accept because there is no connection — recvfrom returns the sender's address for the reply.

A16. TCP: web, file transfer, DB (every byte, in order). UDP: DNS, streaming, gaming (freshness > completeness). Mechanisms: ACK/retransmit/ordering/window vs none.

Wrap-Up — carry these into Lecture 11

TCP VS UDP TCP = connection, **reliable**, **ordered** bytes. UDP = connectionless, 8-byte header, fast and fire-and-forget. Pick per-app, not "better".

PORTS 0–1023 well-known, up to 49151 registered, rest ephemeral. Socket = **IP:port**; 80/443/53/22 worth memorising.

HANDSHAKE & TEARDOWN SYN → SYN-ACK → ACK opens (seq x, x+1, y, y+1); FIN x2 each direction closes. TIME_WAIT = 2×MSL.

RELIABILITY ENGINE ACKs, retransmit (fast on 3 dup-ACKs), flow window from the receiver, congestion window from the network.

NEXT **Lecture 11 — NAT, DHCP & Local Networks:** how your home network actually joins the Internet — private IPs, DORA, and the NAT table in your router.